BUILD A BLOG WRITING CREW WITH CREWAI

By Emmanuel Apilado | October 2026



Introducing Today's Project!

In this project, I'm going to build AI Agents that will produce a polished blog post about any topic using CrewAI.
The multi-agent system has their own specialty according to their role as a Researcher, a Writer and an Editor.

I'm interested in this because this can help me build my own multi-agent system in my future projects related to my work as a DevOps Engineer.

Key tools and concepts

The key tools and concepts I used includes:
- Python 3.12.3 as the programming language
- CrewAI as the Python Framework
- `gemini-3.5-flash-lite` as the AI model
- uv as a package manager
- Google AI Studio to generate API key to use the Gemini model.

Challenges and wins

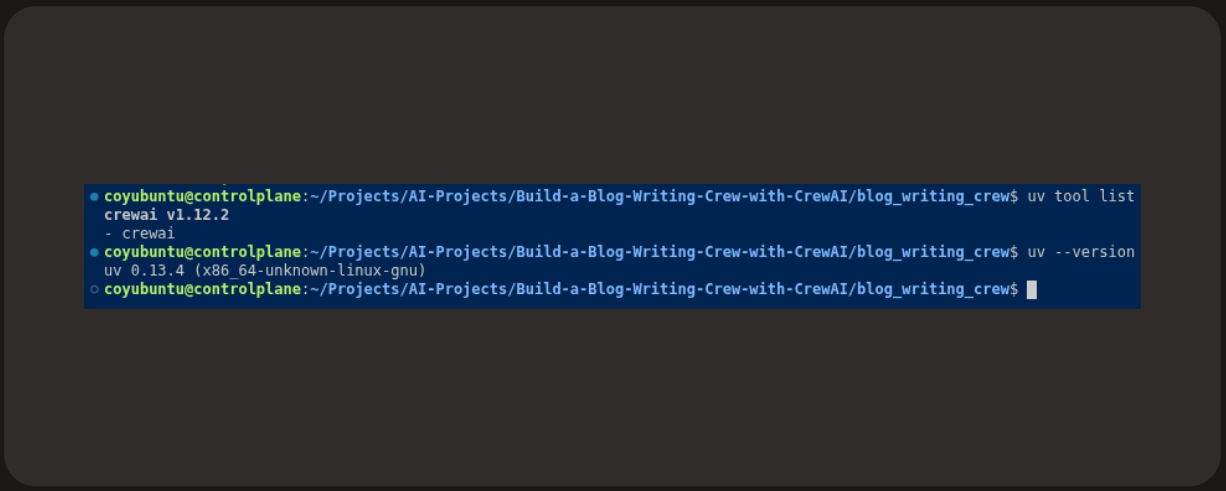
This project took me approximately half a day in a way that I also learned the code, structure, concept moreso how everything is wired and blends together. The most challenging part was just distractions as it interrupts my train of thought as I'm building and studying at the same time -that's it only.

Personal reflection

I did this project today to learn how to use CrewAi its concept along with other tools needed to build for this project. Another skill I want to learn is RAG or MCP.

Installing Python and CrewAI

In this step, I will install Python, a package manager called uv and the CrewAI CLI. Then grab free API key from Google to use Gemini as the brain behind our agents.

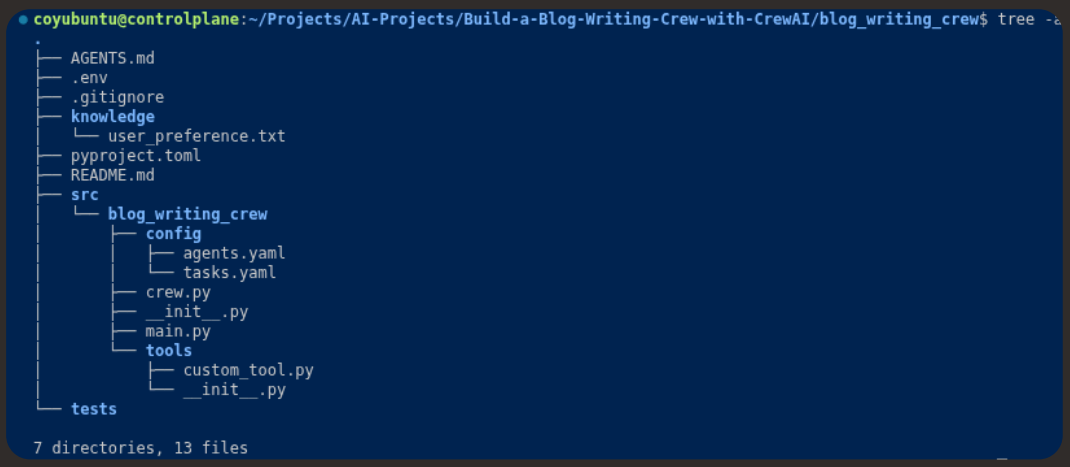


Understanding the uv package manager

I learned that uv is a fast python package manager built in Rust. I replaces older tools like pip and virtualenv with a single, much faster tool. CrewAI uses uv under the hood to manage our projects dependecies and virtual environment.

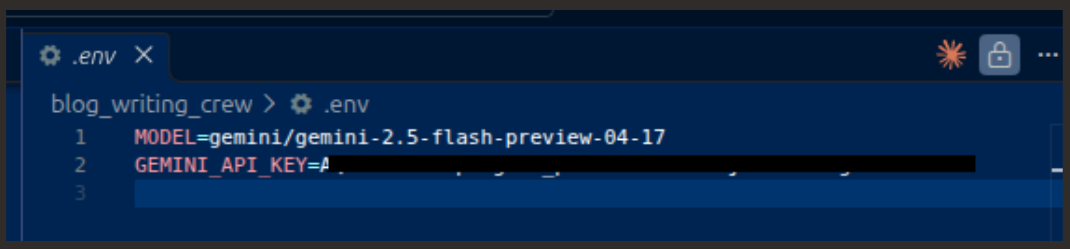
Scaffolding the CrewAI Project

In this step, we'll need a project structure for our blog writing crew. We'll:
- Create a new CrewAI project using the CLI.
- Explore the project structure.
- Configure our API key and model settings.



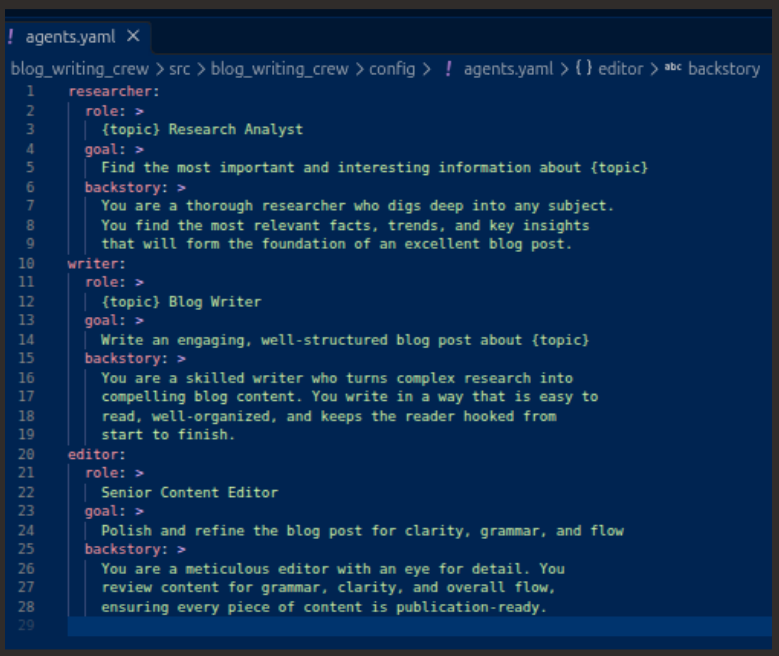
Key project files

Two key files are:
`agents.yaml` which defines who (their roles) the agents are and,
`tasks.yaml` which defines each agents tasks or what they will do.



Configuring AI Agents

In this step, I'm going to define our agents by configuring each agent with their roles, goals and backstory in the `agent.yaml` file.

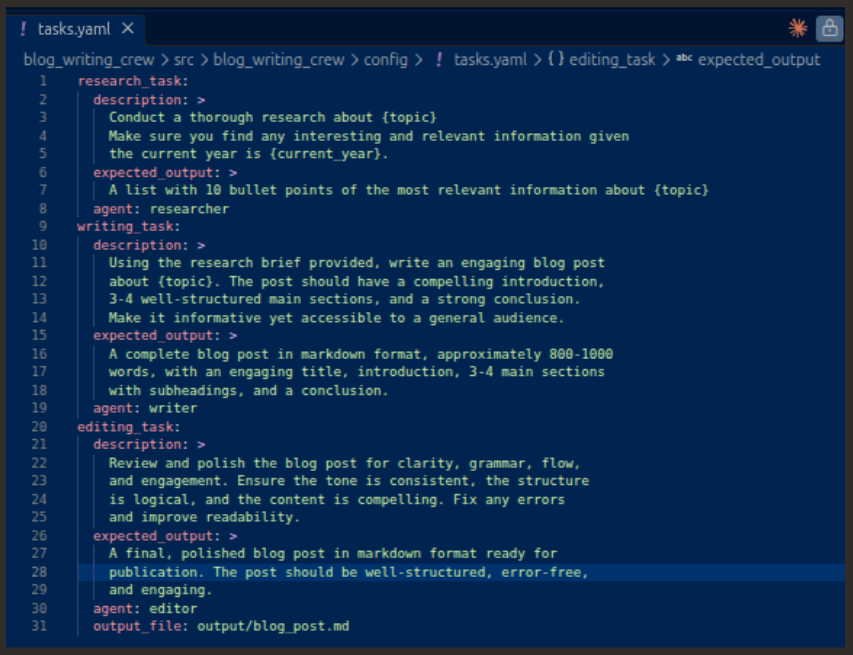


How agent configuration shapes AI behavior

The role is the job title of each agent, the goal is what they need to aim for the outcome and the backststory gives them a persona and expertise to draw fromwhen generating responses.

Defining the Task Pipeline

In this step, I'm defining the tasks for each agent in `tasks.yaml`. These tasks are important because these are set of instructions that I will configure for the agents respectively chaining them together:
1. Research the topic
2. Write a blog post.
3. Edit and polish the final draft

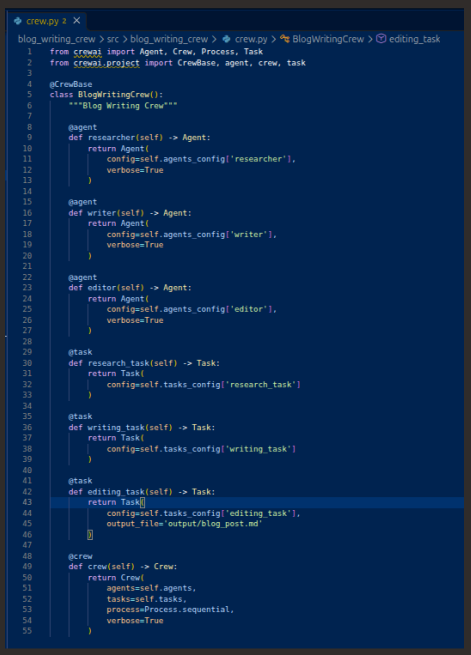


How the tasks chain together

The tasks are chain together by passing the result of the first task (a research brief) by the Researcher agent on to the second task to turn that brief into a blog post by the Writer agent and finally polished the blog post by the Editor agent for the final post and saves it to output/blog_post.md file. Only the Editor agent has the output_file field as the last task needs this because the earlier outputs are passed directly to the next agent in memory.

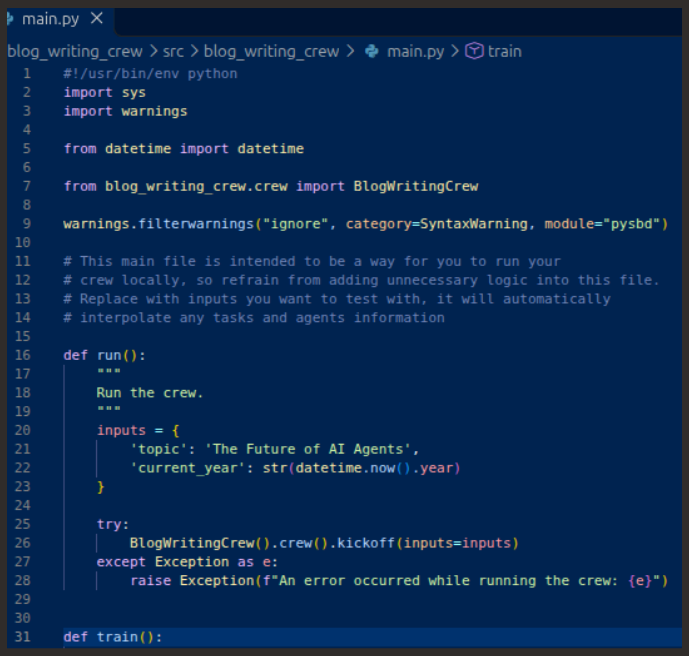
Connecting YAML Configuration to Python

I'm connecting the 3 agents defined in the `agents.yaml` and 3 tasks defined in `tasks.yaml` and turn it into a "working crew" using a Python file named `crew.py`. This will tell the CrewAI framework to read those configurations and create the actual objects and run them.



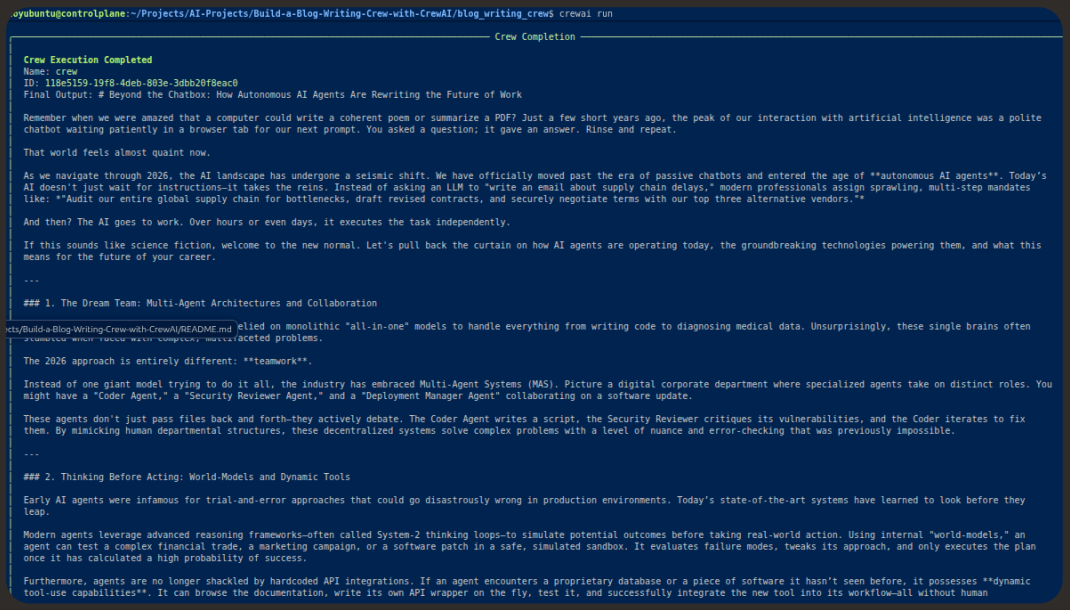
Why method names must match YAML keys

The method names in `crew.py` need to match the keys in my YAML files because these methods creates a Python object from my YAML files. It pulls the keys (researcher, writer, editor) from the YAML file when creating a Python object (an Agent object) that is why the method names should match the keys in my YAML file.


main.py
blog_writing_crew > src > blog_writing_crew > main.py > train
#!/usr/bin/env python
import sys
import warnings
from datetime import datetime
from blog_writing_crew.crew import BlogWritingCrew
warnings.filterwarnings("ignore", category=SyntaxWarning, module="pysbd")
This main file is intended to be a way for you to run your
crew locally, so refrain from adding unnecessary logic into this file.
Replace with inputs you want to test with, it will automatically
interpolate any tasks and agents information
def run():
"""
Run the crew.
"""
inputs = {
'topic': 'The Future of AI Agents',
'current_year': str(datetime.now().year)
}
try:
BlogWritingCrew().crew().kickoff(inputs=inputs)
except Exception as e:
raise Exception(f"An error occurred while running the crew: {e}")
def train():

Running the Blog Writing Crew

In this step, I'll verify CrewAI framework knows how to talk to Google Gemini as this is the language model we selected. I'll also install all the project dependencies that CrewAI scaffolded project requires. We'll:
- Install dependencies and run our crew.
- Watch your agents collaborate to produce a blog post.
- Experiment with a different topic.



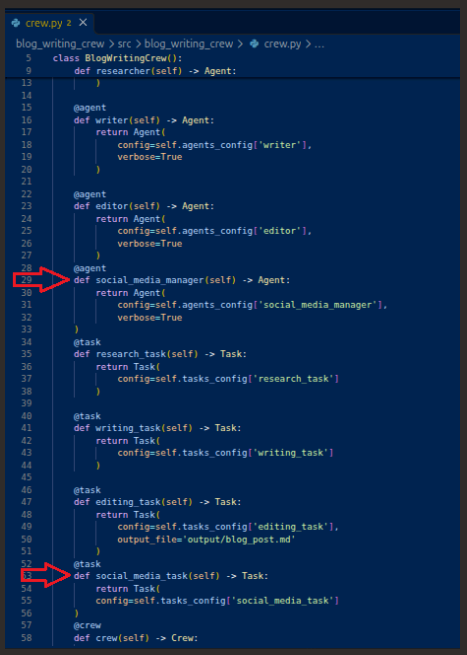
How the agents collaborated

I ran my crew and the agents collaborated sequentially by passing the result/output from the researcher -> writer -> editor agents and perform their tasks from the given output.


blog_post.md
blog_writing_crew > output > blog_post.md > # Beyond the Chatbox: How Autonomous AI Agents Are Rewriting the Future of Work
How to Learn Programming as a Beginner: The Modern Guide
Remember when learning to code meant spending your entire first weekend fighting with installation wizards, path variables, and notoriously stubborn text editors just to print "Hello, World!"?
Thankfully, those days are behind us.
Stepping into the programming world today looks entirely different than it did just a few years ago. With artificial intelligence acting as a native co-pilot, cloud-based environments eliminating setup headaches, and a massive shift away from rote memorization, there has never been a more exciting—or accessible—time to learn how to code.
Whether you want to build the next big app, automate your daily workflow, or completely pivot your career, here is your blueprint for learning programming as a beginner.
---
1. Ditch the Local Setup Friction and Start in the Browser
In the past, the absolute biggest drop-off point for beginners wasn't difficult logic loops or complex math—it was environment setup. Trying to configure a local Integrated Development Environment (IDE) before writing a single line of code was enough to make anyone quit.
Today, cloud-based environments have completely removed that barrier to entry.
Tools like GitHub Codespaces, Replit, and StackBlitz allow you to spin up a fully functioning, professional development environment instantly inside a browser tab with zero local configuration overhead. You can go from zero to writing your first lines of code in under ten seconds.
Your action step: Don't waste time downloading heavy software right now. Pick a cloud-based IDE, open a browser tab, and start experimenting immediately. Lower the friction between your curiosity and the keyboard.
---
2. Embrace AI as Your Native Learning Tool (Not a Cheat Code)
There's a common misconception that using AI tools like GitHub Copilot or specialized LLM coding tutors is "cheating." In modern software development—and modern learning—nothing could be further from the truth.
Artificial intelligence is integrated into the beginner workflow from day one. Instead of wasting hours memorizing rigid syntax or combing through outdated forum threads, modern beginners learn **prompt engineering for code**.
You are no longer expected to be a human encyclopedia. Instead, your job is to learn how to:
* Read and understand AI-generated code.
* Guide AI models to refactor or optimize solutions.
* Practice **prompt-driven debugging**, which means feeding error logs effectively into AI models, interpreting the explanations, and verifying the corrections through an interactive, Socratic dialogue.
AI handles the boilerplate code and basic functions instantly, freeing you up to focus on what humans do best: high-level logic, creativity, and system problem-solving.
---
3. Focus on Computational Thinking and Choose Your First Language Wisely
When you're starting out, it's easy to get paralyzed by the sheer number of programming languages out there. Should you learn Python? JavaScript? Rust?
The secret that experienced developers know is that **the specific language matters far less than mastering core computational concepts**. Variables, loops, conditional logic, object-oriented principles, and asynchronous programming are universal. Once these mental models are internalized, pivoting between languages becomes a breeze.
However, starting with the right tool for your goals still matters:
* **Python** remains the undisputed entry point if you are interested in data science, AI orchestration, and task automation.
* **JavaScript and TypeScript** continue to anchor web development and cross-platform applications.

Adding a Social Media Manager Agent

In this project extension, I'm adding a fourth agent, a Social Media Manager agent to that turns polished blog posts into Twitter/X threads and LinkedIn posts where this will simulate how companies use multi-agent AI systems to automate content.



Extending the crew with new agents and tasks

I added the social media manager by creating Social Media Manager agent in agents.yaml, then added a task for the Social Media Manager agent in tasks.yaml, and finally added the wiring for this agent in crew.py.



Social media output

In this project extension, my social media agent generated a posts where it clearly outlined social media contents for Twitter thread and the LinkedIn Post Summary. It differs from the original blog posts as the structure of the original post is more of a "blog" concept where the Social Media Manager agent uses that polished blog to create a socia media content for the Twitter/X and LinkedIn social media platforms where it highlights key points and created a summary of the main takeaways in a professional tone.

Experimenting with agent backstories

When I changed the backstory of the Social Media Manager to its defined persona, the tone of the output changed. ... This showed me that the difference in tone shows how much the backstory influences our agent's output. This is a powerful way to customize agent behavior without changing any code.